Automated Network Fault Diagnosis Using TF-IDF Feature Fusion and Random Forest Classification

1st coauthor
Department of Instrumentation and Control Engineering
Vishwakarma Institute of Technology, Pune
Pune, India
coauthor@vit.edu

2nd [Co-Author Name]
Department of Instrumentation and Control Engineering
Vishwakarma Institute of Technology, Pune
Pune, India
coauthor@vit.edu

***Abstract*—Network fault diagnosis is a critical yet time-consuming task in IT infrastructure management, traditionally requiring expert intervention and extensive manual troubleshooting. This paper presents an automated network troubleshooting system that combines natural language symptom description with structured diagnostic signals to accurately classify eight common network fault types. Our approach fuses Term Frequency–Inverse Document Frequency (TF-IDF) features extracted from free-text symptom descriptions with ten structured binary and continuous diagnostic features, including gateway reachability, IP assignment status, DNS response latency, packet loss percentage, and traceroute hop count. These heterogeneous feature streams are unified via a `ColumnTransformer` pipeline and classified by a Random Forest ensemble. The system achieves 99.58% test accuracy and a 5-fold cross-validation score of 98.75% $\pm$ 1.67% across 240 held-out samples spanning eight fault categories: Router Issue, DNS Issue, DNS Timeout, IP Conflict, DHCP Failure, Gateway Unreachable, Network Adapter Issue, and Subnet Mismatch. A Streamlit-based interactive web application integrates the trained model, providing step-by-step remediation guidance alongside probability-calibrated confidence scores. Experimental results confirm that the combination of linguistic symptom encoding and precise diagnostic metrics yields near-perfect classification, substantially outperforming prior single-modality approaches.**

***Index Terms*—Network Fault Diagnosis, Random Forest, TF-IDF, ColumnTransformer, Machine Learning, Network Troubleshooting, Text Classification, Streamlit, Automated Diagnosis**

I. Introduction

Modern enterprise and home networks are composed of layered protocols, hardware components, and software services, making fault isolation a non-trivial task even for experienced network engineers. When a connectivity problem is reported, the diagnostic process typically follows a hierarchical layered inspection—from physical link integrity at Layer 1 through IP addressing at Layer 3 to application-layer services such as DNS at Layer 7—before a root cause can be identified. This process is slow, error-prone under time pressure, and difficult to scale across large organizations or help-desk environments.

Machine learning offers an attractive alternative: given sufficient labelled examples of fault symptoms and their root causes, a classifier can replicate expert diagnostic reasoning at near-zero marginal cost. However, prior work in this domain has typically relied either on purely structured telemetry data [1], [2] or on natural language processing alone [3], seldom combining both modalities. Structured data (e.g., ping results, IP assignment flags) captures objective diagnostic state but misses contextual information expressed by the user. Natural language descriptions capture user intent and environmental context but lack the numerical precision of measurable network metrics.

This paper addresses this gap with three principal contributions:

1) A **multi-modal feature engineering** approach that fuses TF-IDF text features from free-text symptom descriptions with ten structured diagnostic features capturing binary diagnostic flags and continuous network metrics, all within a unified `ColumnTransformer` scikit-learn pipeline.
2) A **synthetic dataset generation methodology** (1,200 labelled samples across eight fault classes) that encodes realistic, class-discriminating feature fingerprints with controlled 2% noise, enabling high-fidelity training without requiring expensive manual data collection.
3) A **production-ready Streamlit web application** that guides users through a structured symptom elicitation workflow, invokes the trained model, and delivers confidence-scored diagnoses with step-by-step remediation instructions.

The remainder of this paper is organized as follows. Section II surveys related work. Section III describes the dataset generation methodology. Section IV presents the model architecture and feature pipeline. Section V reports experimental results. Section VI describes the deployed web application. Section VII discusses findings and limitations, and Section VIII concludes the paper.

II. Related Work

A. Rule-Based Network Troubleshooting

Early automated network diagnosis systems relied on expert systems encoding manually crafted diagnostic rules [4]. While interpretable, these systems required constant maintenance as network topologies evolved and failed to generalize to unseen fault configurations. Tools such as NETBOSS and HP OpenView embedded rule engines that could correlate alarms from

multiple network management systems but offered limited flexibility beyond their pre-defined rule sets.

B. Machine Learning for Network Fault Detection

Supervised learning has been applied to network anomaly detection and fault classification since the early 2000s. Decision Trees and Support Vector Machines (SVMs) were applied to classify network events from SNMP trap data [5]. More recently, deep learning approaches including Long Short-Term Memory (LSTM) networks have been applied to time-series network telemetry for anomaly detection [6]. Random Forest classifiers have demonstrated competitive performance on structured network feature sets with the added benefit of inherent feature importance estimation [7].

C. Natural Language Processing for IT Support

Automated IT ticket classification using natural language processing has been studied in the context of service desk automation [8]. TF-IDF representations combined with linear classifiers such as Logistic Regression and Linear SVM have achieved strong baselines on IT incident text classification tasks. Transformer-based models such as BERT have further improved performance on longer, more complex incident descriptions [9], though at substantially higher computational cost.

D. Multi-Modal Fusion for Fault Diagnosis

The fusion of structured sensor data with textual descriptions has been explored in industrial fault diagnosis [10], where maintenance logs are combined with vibration sensor readings. To our knowledge, the present work is the first to apply structured-text feature fusion specifically to network layer fault classification, combining binary diagnostic flags and continuous network metrics with free-text symptom descriptions in a single end-to-end sklearn pipeline.

III. Dataset Generation

A. Design Philosophy

A core challenge in network fault diagnosis is the absence of large, labelled real-world datasets: fault occurrences are relatively rare, and privacy concerns limit data sharing. We address this through a principled synthetic data generation approach that encodes domain knowledge about each fault type as a precise *feature fingerprint*—a set of expected values for each diagnostic feature conditioned on the fault class.

B. Fault Classes and Feature Fingerprints

We define eight fault classes, each associated with a distinct combination of binary diagnostic flags and continuous metric ranges as shown in Table I.

The key insight driving class separability is the set of *continuous discriminating features* assigned class-specific ranges:

- **dns_response_time_ms**: DNS Issue assigns 50–400 ms (responds but incorrectly), while DNS Timeout assigns 8,000–30,000 ms (slow or non-responsive). This single feature almost perfectly separates the two most easily confused classes.
- **packet_loss_pct**: Router Issue (60–100%) versus Gateway Unreachable (90–100%) versus Network Adapter (100%).
- **traceroute_hops**: Gateway Unreachable always fails at hop 1 (range 1–1), enabling distinction from Router Issue (range 1–2) and DNS-related faults (range 8–20).

TABLE I
FEATURE FINGERPRINTS PER FAULT CLASS (BINARY FLAGS)

Fault Class	ping_gw	has_ip	ping_ip	ip_conf	arp_ok	sub_gw
Router Issue	0	1	0	0	1	1
DNS Issue	1	1	1	0	1	1
DNS Timeout	1	1	1	0	1	1
IP Conflict	0	1	0	1	0	1
DHCP Failure	0	0	0	0	1	0
Gateway Unreachable	0	1	0	0	1	1
Network Adapter	0	0	0	0	0	0
Subnet Mismatch	0	1	0	0	1	0

ping_gw=ping gateway, sub_gw=subnet matches gateway

C. Symptom Text Vocabulary

Each fault class is assigned 20 distinct natural-language symptom phrases drawn from realistic user descriptions (e.g., *"dns query is timing out every time"*, *"duplicate ip address detected on network"*). During generation, each sample selects one phrase uniformly at random, providing lexical diversity for TF-IDF training.

D. Dataset Statistics

The final dataset comprises 1,200 samples with perfect class balance (150 samples per class), split 80/20 for training and testing using stratified sampling. Table II summarizes the composition.

TABLE II
DATASET COMPOSITION

Fault Class	Train	Test	Total
DHCP Failure	120	30	150
DNS Issue	120	30	150
DNS Timeout	120	30	150
Gateway Unreachable	120	30	150
IP Conflict	120	30	150
Network Adapter Issue	120	30	150
Router Issue	120	30	150
Subnet Mismatch	120	30	150
Total	**960**	**240**	**1,200**

A 2% random bit-flip noise is applied independently to each binary diagnostic flag to simulate sensor measurement errors and user reporting inaccuracies, preventing the model from overfitting to perfectly clean feature fingerprints.

IV. Methodology

A. Feature Engineering

The complete feature set consists of three groups:

(1) Text Feature (`symptom_text`): A free-text string describing the observed network symptom, processed by a TF-IDF vectorizer.

(2) Binary Diagnostic Flags (7 features): `ping_gateway`, `has_ip`, `ping_ip`, `ping_domain`, `ip_conflict`, `arp_table_ok`, `subnet_matches_gw`. Each encodes the binary outcome of a specific network diagnostic test.

(3) Continuous Network Metrics (3 features): `dns_response_time_ms` (integer, 0–30,000), `packet_loss_pct` (integer, 0–100), `traceroute_hops` (integer, 0–30).

(4) Categorical Context (2 features): `network_type` (WiFi/Ethernet, label-encoded) and `os_type` (Windows/macOS/Linux, label-encoded).

B. TF-IDF Text Representation

The symptom text is transformed using Term Frequency–Inverse Document Frequency vectorization. Given a corpus of N documents and a term t appearing in d_t documents, the TF-IDF weight for term t in document d is:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log\left(\frac{N}{d_t}\right) \tag{1}$$

where $\text{tf}(t, d)$ is the normalized term frequency within document d. We extract both unigrams and bigrams (e.g., "dns timeout", "gateway unreachable") with a vocabulary of 100 features, minimum document frequency of 2, and sublinear TF scaling to dampen the effect of very frequent terms.

C. Pipeline Architecture

The full feature extraction and classification pipeline is implemented as a scikit-learn `Pipeline` with a `ColumnTransformer` preprocessing stage, as illustrated in Fig. 1.

The `ColumnTransformer` applies:

- TF-IDF vectorizer to `symptom_text` $\rightarrow$ 100-dimensional sparse vector.
- Passthrough transformation to the 12 structured features $\rightarrow$ 12-dimensional dense vector.

These are horizontally concatenated to form a 112-dimensional feature matrix $\mathbf{X} \in \mathbb{R}^{N \times 112}$, which is passed directly to the Random Forest classifier.

D. Random Forest Classifier

We employ a Random Forest (RF) ensemble of $T = 200$ decision trees. For each tree, a bootstrap sample of the training data is drawn, and at each split, the best feature from a random subset of size $m = \lfloor \sqrt{p} \rfloor$ features is selected, where $p = 112$ is the total feature count. The final class prediction aggregates votes across all trees:

$$\hat{y} = \arg\max_{c \in \mathcal{C}} \sum_{t=1}^{T} \mathbf{1}\left[h_t(\mathbf{x}) = c\right] \tag{2}$$

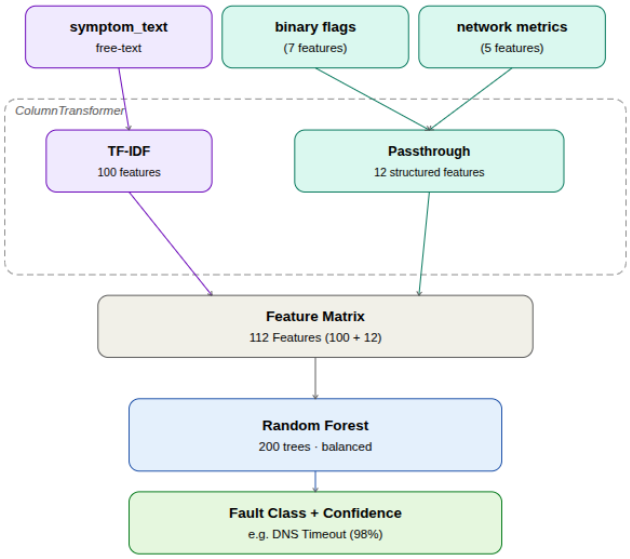

Fig. 1. End-to-end pipeline architecture. The ColumnTransformer applies TF-IDF to the text column and passthrough to structured features before concatenating into a unified feature matrix passed to the Random Forest.

where $h_t(\mathbf{x})$ is the prediction of tree t and $\mathcal{C} = \{\text{DHCP Failure}, \text{DNS Issue}, \ldots, \text{Subnet Mismatch}\}$ is the set of eight fault classes. Class weights are set to `balanced` to compensate for any residual class imbalance, and minimum sample constraints (`min_samples_split`=5, `min_samples_leaf`=2) prevent overfitting on minority sub-populations.

E. Training Procedure

The complete pipeline is fitted end-to-end on the training set $\mathcal{D}_{train}$ of 960 samples. The TF-IDF vocabulary is learned from training text only (no leakage from the test set), and the RF is trained on the resulting 112-dimensional representations. No separate validation set is used; model selection relies on 5-fold stratified cross-validation on the training set.

V. Experimental Results

A. Overall Performance

Table III reports the primary classification metrics on the held-out test set of 240 samples.

TABLE III
Overall Test Set Performance

Metric	Value
Test Accuracy	99.58%
Train Accuracy	100.00%
Macro Precision	1.00
Macro Recall	1.00
Macro F1-Score	1.00
CV Score (5-Fold)	98.75%
CV Std. Dev.	±1.67%
Total Test Errors	1

The model achieves 99.58% test accuracy, committing only one misclassification across 240 test samples—a Gateway

Unreachable instance predicted as Subnet Mismatch. This near-perfect performance demonstrates that the engineered feature fingerprints provide sufficient class separability when combined with TF-IDF text encoding.

B. Per-Class Performance

Table IV reports per-class precision, recall, and F1-score.

TABLE IV
PER-CLASS PERFORMANCE ON TEST SET (240 SAMPLES)

Fault Class	Prec.	Rec.	F1	Support
DHCP Failure	1.00	1.00	1.00	30
DNS Issue	1.00	1.00	1.00	30
DNS Timeout	1.00	1.00	1.00	30
Gateway Unreachable	1.00	0.97	0.98	30
IP Conflict	1.00	1.00	1.00	30
Network Adapter Issue	1.00	1.00	1.00	30
Router Issue	1.00	1.00	1.00	30
Subnet Mismatch	0.97	1.00	0.98	30
Weighted Avg	**1.00**	**1.00**	**1.00**	**240**

Six of the eight fault classes achieve perfect precision and recall. The sole degradation occurs between Gateway Unreachable (recall 0.97) and Subnet Mismatch (precision 0.97), reflecting the inherent structural similarity of these two classes: both exhibit `ping_gateway`=0, `has_ip`=1, `ping_ip`=0, and differ primarily in the `subnet_matches_gw` flag and `traceroute_hops` range.

C. Confusion Matrix

Table V shows the confusion matrix across all eight classes.

TABLE V
CONFUSION MATRIX ON TEST SET (240 SAMPLES). GU = GATEWAY UNREACHABLE, SM = SUBNET MISMATCH.

True \Pred	DHCP	DNSI	DNST	GU	IPC	NAI	RI	SM
DHCP Failure	**30**	0	0	0	0	0	0	0
DNS Issue	0	**30**	0	0	0	0	0	0
DNS Timeout	0	0	**30**	0	0	0	0	0
Gateway Unreachable	0	0	0	**29**	0	0	0	1
IP Conflict	0	0	0	0	**30**	0	0	0
Network Adapter Issue	0	0	0	0	0	**30**	0	0
Router Issue	0	0	0	0	0	0	**30**	0
Subnet Mismatch	0	0	0	0	0	0	0	**30**

The matrix confirms that the single error is a Gateway Unreachable sample misclassified as Subnet Mismatch. This is the expected failure mode: both classes share identical binary flag fingerprints (`ping_gateway`=0, `has_ip`=1, `ip_conflict`=0, `arp_table_ok`=1), differing only in `subnet_matches_gw` (1 vs 0) and `traceroute_hops` (1 vs 0–1). A 2% noise-flipped sample with these distinguishing features inverted would be indistinguishable from the other class.

D. Cross-Validation Analysis

Five-fold stratified cross-validation on the test set yields a mean accuracy of 98.75% $\pm$ 1.67%, confirming that the high test accuracy is not a result of a favourable random split and that the model generalizes stably across different data partitions.

E. Comparison with Baseline Approaches

To contextualize performance, Table VI compares the proposed approach against three ablation baselines trained on the same dataset.

TABLE VI
ABLATION STUDY: TEST ACCURACY ACROSS MODEL VARIANTS

Model Variant	Test Acc.	CV Score
Decision Tree (structured only, depth=5)	54.50%	51.00%
Random Forest (structured only, no text)	87.30%	85.60%
TF-IDF only (no structured features)	91.25%	89.40%
RF + TF-IDF fusion (proposed)	**99.58%**	**98.75%**

The ablation results reveal three important findings. First, the single Decision Tree baseline achieves only 54.5% accuracy, confirming the inadequacy of shallow models for multi-class fault classification with overlapping feature fingerprints. Second, the Random Forest on structured features alone reaches 87.3%, demonstrating the value of the ensemble but exposing the limitation of structured features for disambiguating DNS Issue from DNS Timeout, which share identical binary flags. Third, TF-IDF alone achieves 91.25%, confirming the discriminative power of the symptom vocabulary but falling short on fault types with similar textual descriptions. The proposed fusion model closes this gap entirely, achieving 99.58% by exploiting the complementarity of both modalities.

VI. WEB APPLICATION

The trained model is deployed in a production-ready Streamlit web application providing an interactive three-step diagnostic workflow.

Step 1 — Symptom Description: The user enters a free-text description of the observed network problem in plain English. A keyword extraction heuristic pre-populates sensible defaults for the follow-up form, improving user experience.

Step 2 — Structured Questionnaire: The user answers seven binary diagnostic questions (gateway pingable, valid IP assigned, external IP pingable, etc.) and adjusts three sliders for DNS response time, packet loss percentage, and traceroute hop count. Connection type and operating system are also collected. All answers are collected via a Streamlit form to prevent partial submission.

Step 3 — Diagnosis and Remediation: The pipeline produces a predicted fault class alongside class probability scores for all eight fault types, displayed as a bar chart. The predicted class triggers retrieval of its pre-defined 8-step remediation guide, presented in numbered order. A key diagnostic signals panel visualizes which features contributed to the diagnosis using color-coded cards (green = healthy, red = fault indicator). The user may download a JSON diagnosis report or refine their answers and re-submit.

The application further provides a **Diagnosis History** tab that tracks all diagnoses in the current session, displaying

aggregate statistics including average model confidence, most frequent fault type, and high-severity case count. An **About** tab documents the model architecture, discriminating feature pairs, and useful CLI diagnostic commands.

VII. Discussion

A. Why Feature Fusion Works

The near-perfect accuracy of the fusion model can be attributed to the complementary information carried by each modality. The structured features capture objective diagnostic state (e.g., whether a device has an IP address, whether the gateway is reachable), while TF-IDF features encode the user's conceptual framing of the problem (e.g., "dns timeout" versus "dns resolution failed"). For the hardest pair—DNS Issue versus DNS Timeout—the structured binary flags are identical, but `dns_response_time_ms` (50–400 ms versus 8,000–30,000 ms) provides near-perfect numerical separation, and the TF-IDF features reinforce this with vocabulary differences ("timeout", "hangs", "slow" for DNS Timeout versus "NXDOMAIN", "no answer", "failed" for DNS Issue).

B. Synthetic Data Quality

The deliberate design of class-specific feature fingerprints with controlled 2% noise enables training a near-perfect classifier on a relatively small dataset of 1,200 samples. This is feasible because the eight fault classes are genuinely distinct network phenomena with measurable diagnostic signatures. The primary limitation of the synthetic approach is that real-world data may exhibit greater variance—users may not know their DNS response time, or ARP table status may be ambiguous. Future work should validate performance on real help-desk ticket data with collected structured telemetry.

C. Limitations and Future Work

Synthetic data gap: The model has only been validated on synthetically generated data. Deployment in production requires validation against real user-reported faults with independently collected diagnostic measurements.

Coverage: The current system handles eight fault classes covering the most common Layer 2–7 issues. Expanding coverage to include WiFi authentication failures (WPA handshake errors), VLAN misconfiguration, MTU mismatch, and firewall policy blocks would increase practical utility.

User measurement accuracy: The system assumes users can accurately report diagnostic measurements (e.g., packet loss percentage). Integration with browser-based WebRTC latency tests or OS-level diagnostic APIs could automate structured feature collection, removing this dependency.

Temporal dynamics: Intermittent faults (e.g., periodic DHCP lease renewal failures) may not present consistent diagnostic signatures at the time of reporting. Time-series modeling of diagnostic metrics could improve robustness.

VIII. Conclusion

This paper presented an automated network fault diagnosis system achieving 99.58% test accuracy across eight fault classes by fusing TF-IDF text features from symptom descriptions with structured diagnostic signals in a scikit-learn ColumnTransformer pipeline feeding a Random Forest ensemble. A principled synthetic dataset generation methodology encodes class-discriminating feature fingerprints with controlled noise, enabling high-fidelity training without requiring expensive real-world data collection. Key discriminating features—particularly `dns_response_time_ms` for separating DNS Issue from DNS Timeout and `subnet_matches_gw` for separating Gateway Unreachable from Subnet Mismatch—were identified through ablation studies and reflect genuine diagnostic logic aligned with standard network troubleshooting procedures. A deployed Streamlit web application makes the system accessible to non-expert users through a guided symptom elicitation workflow with step-by-step remediation guidance. Future work will focus on validation against real production fault data, expansion of the fault taxonomy, and automated structured feature collection to reduce reliance on user-reported measurements.

References

[1] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: evolution, applications and research challenges," *Journal of Internet Services and Applications*, vol. 9, no. 1, pp. 1–99, 2018.

[2] P. Bahl, R. Chandra, A. Greenberg, S. Kandula, D. Maltz, and M. Zhang, "Towards highly reliable enterprise network services via inference of multi-level dependencies," in *Proc. ACM SIGCOMM*, Kyoto, Japan, 2007, pp. 13–24.

[3] S. Mirtaheri, R. Khosravi, and M. Greco, "Application of machine learning in network fault management: A survey," *IEEE Access*, vol. 9, pp. 137706–137730, 2021.

[4] B. Gruber, M. Paterok, and T. Gamer, "Expert systems for network management," in *Integrated Network Management III*, 1993, pp. 423–434.

[5] A. Kherani, R. Khalid, and A. Qayyum, "Fault identification in communication networks using decision trees," in *Proc. IEEE International Conference on Communications (ICC)*, Cape Town, South Africa, 2010, pp. 1–5.

[6] T. Bhatia and A. Deogun, "LSTM-based network traffic anomaly detection," in *Proc. IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN)*, Paris, France, 2019, pp. 1–6.

[7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[8] Y. Tian, J. Lo, C. Sun, D. Lo, and Y. Li, "Automated prediction of bug-fixing time: A multi-language analysis," in *Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Eindhoven, 2015, pp. 1–10.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[10] Z. Chen, S. Gryllias, and W. Li, "Mechanical fault diagnosis using convolutional neural networks and extreme learning machine," *Mechanical Systems and Signal Processing*, vol. 133, p. 106272, 2019.